

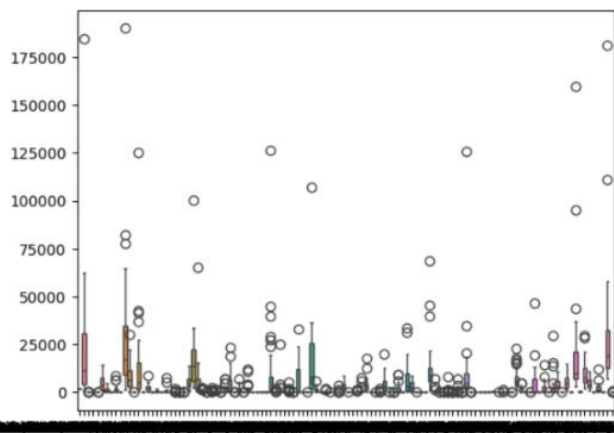
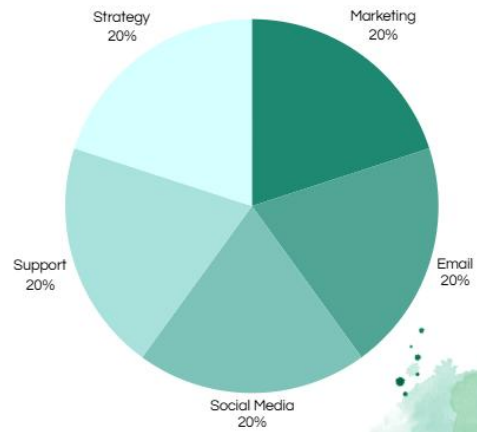
From Quiet to Booming: Yearly M&A Regimes in the UK & Asia, and Beyond

Think of M&A like “big companies eating smaller ones.” We looked at yearly UK deal activity (plus regions like Asia/Africa) to see when the market was busy (many/big deals) vs quiet. First we did quick visuals to spot patterns, then we grouped years into Hot and Cold using an unsupervised method (no labels—just patterns). Next, we turned that into a simple forecast task: use this year’s signals to predict whether next year will be Hot or Cold. The result is an easy story for leaders: which years look strong, which look weak, and a basic heads-up for planning—while remembering the data is small and yearly, so predictions are helpful signals, not guarantees.

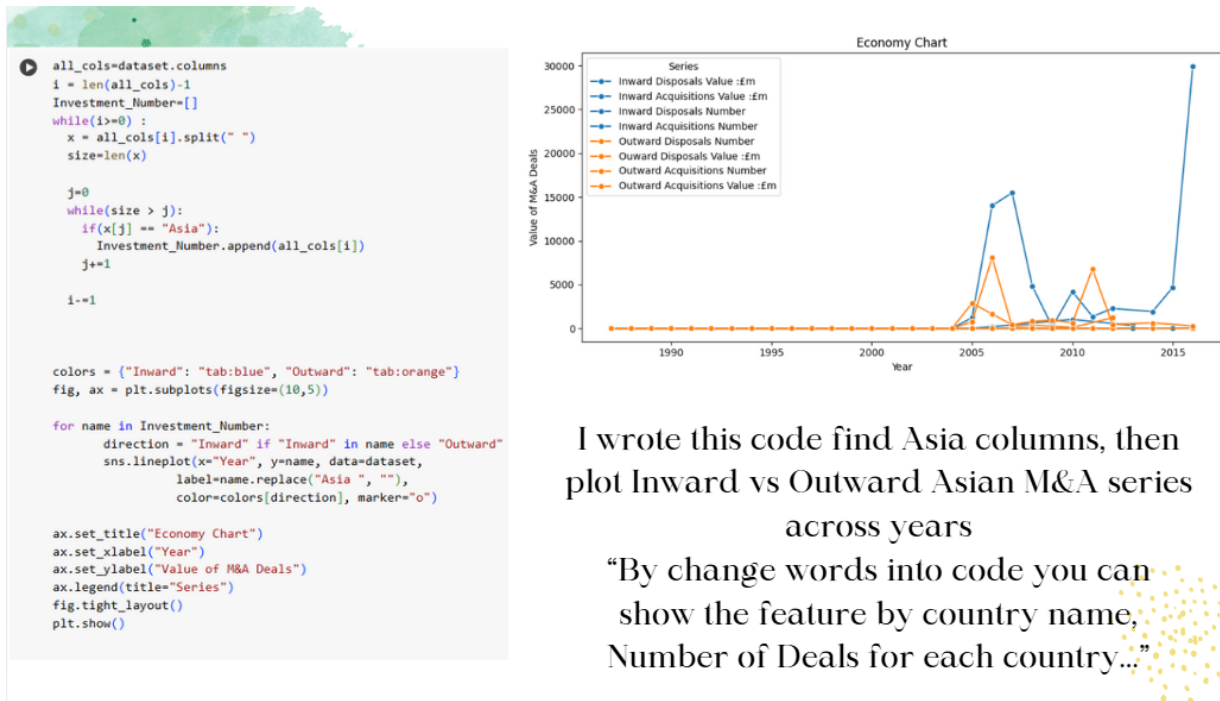
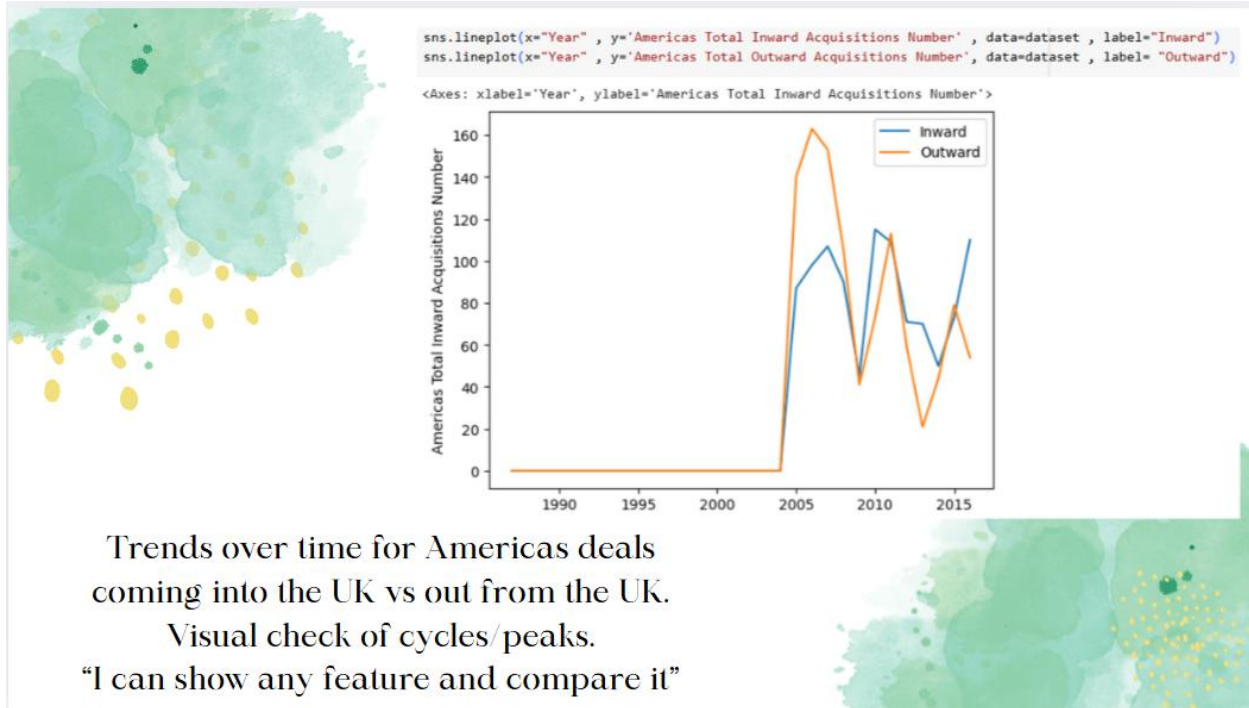


Whole World

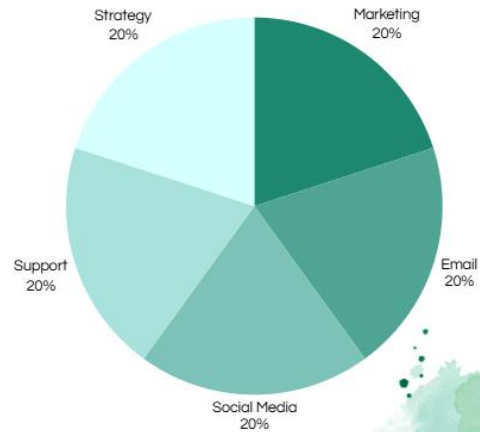
EDA Phase



Quick overview of each column's spread and outliers. Helps spot weird values before modeling.



Clustering Phase



Pick economically important columns by name (values & counts, inward/outward, domestic, + Asia/Africa). This reduces noise before clustering.

```
# --- Build strong_cols FIRST (from your raw dataset) ---
# Regex groups that match the ONS column names
pat_in_val = r"^M&A : Inward : Value of acquisitions"
pat_out_val = r"^M&A : Outward : Value of acquisitions"
pat_dom_val = r"^M&A : Domestic : .*(Value|Total payments)"
pat_in_num = r"^M&A : Inward : Number of acquisitions"
pat_out_num = r"^M&A : Outward : Number of acquisitions"

# Collect columns
in_val_cols = dataset.filter(regex=pat_in_val).columns
out_val_cols = dataset.filter(regex=pat_out_val).columns
dom_val_cols = dataset.filter(regex=pat_dom_val).columns
in_num_cols = dataset.filter(regex=pat_in_num).columns
out_num_cols = dataset.filter(regex=pat_out_num).columns

# Optional regional VALUE totals
asia_val_cols = dataset.filter(regex=r"^Asia .*Acquisitions Value").columns
africa_val_cols = dataset.filter(regex=r"^Africa .*Acquisitions Value").columns

# Master list used below
strong_cols = (list(in_val_cols) + list(out_val_cols) + list(dom_val_cols) +
               list(in_num_cols) + list(out_num_cols) +
               list(asia_val_cols) + list(africa_val_cols))

print("Strong cols found:", len(strong_cols))
```

Strong cols found: 32


```

nan_per_col = dataset[strong_cols].isna().sum().sort_values() # fewest NaNs first
core_cols = nan_per_col.index[:31].tolist()

vary = dataset[core_cols].nunique(dropna=True)
core_cols = [c for c in core_cols if vary[c] > 1]
print(len(core_cols))
print("Core columns:", core_cols)

```

31
Core columns: ['M&A : Domestic : Value of preference & loan stock : £m', 'M&A : Dome:

From the strong set, keep the ~31 least-missing columns and drop columns that barely change. We keep signals that actually vary.

```

X_raw = dataset.loc[:, core_cols].to_numpy(dtype=float)

[1382] from sklearn.impute import KNNImputer
      impute = KNNImputer(n_neighbors=3, weights="distance")
      X_raw = impute.fit_transform(X_raw)

[1383] num_nans = np.isnan(X_raw).sum()
      print("Number of NaNs:", num_nans)

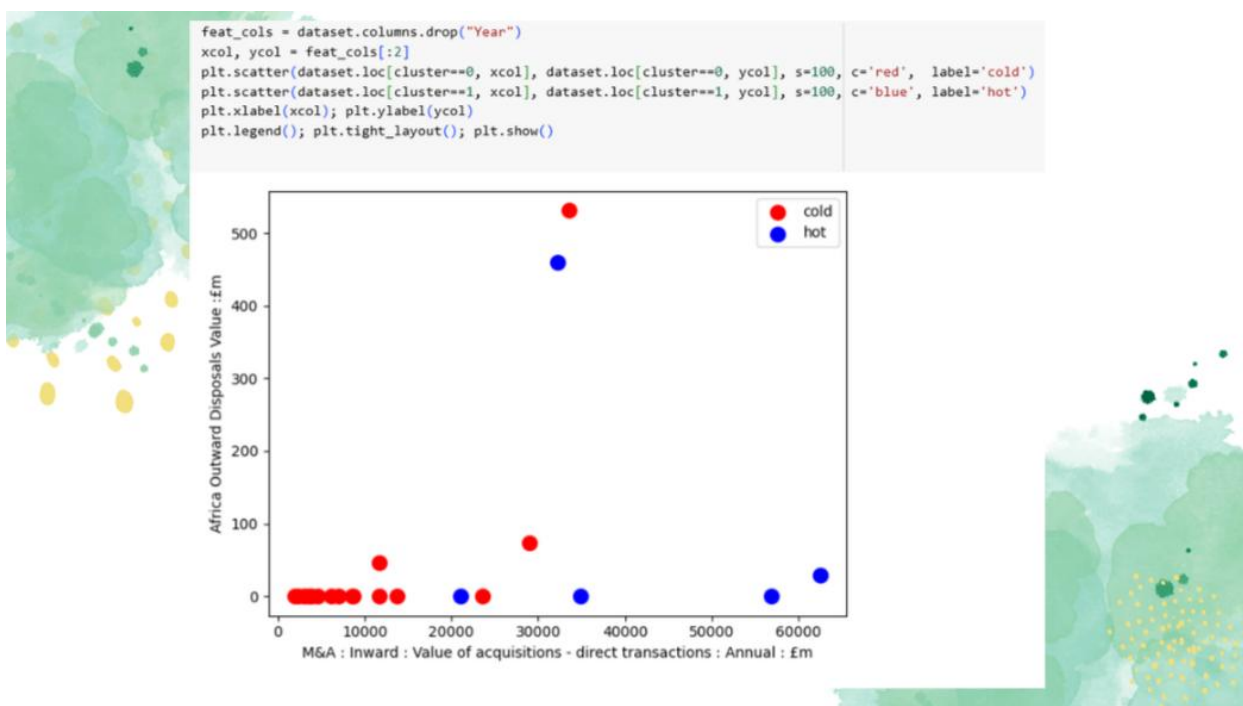
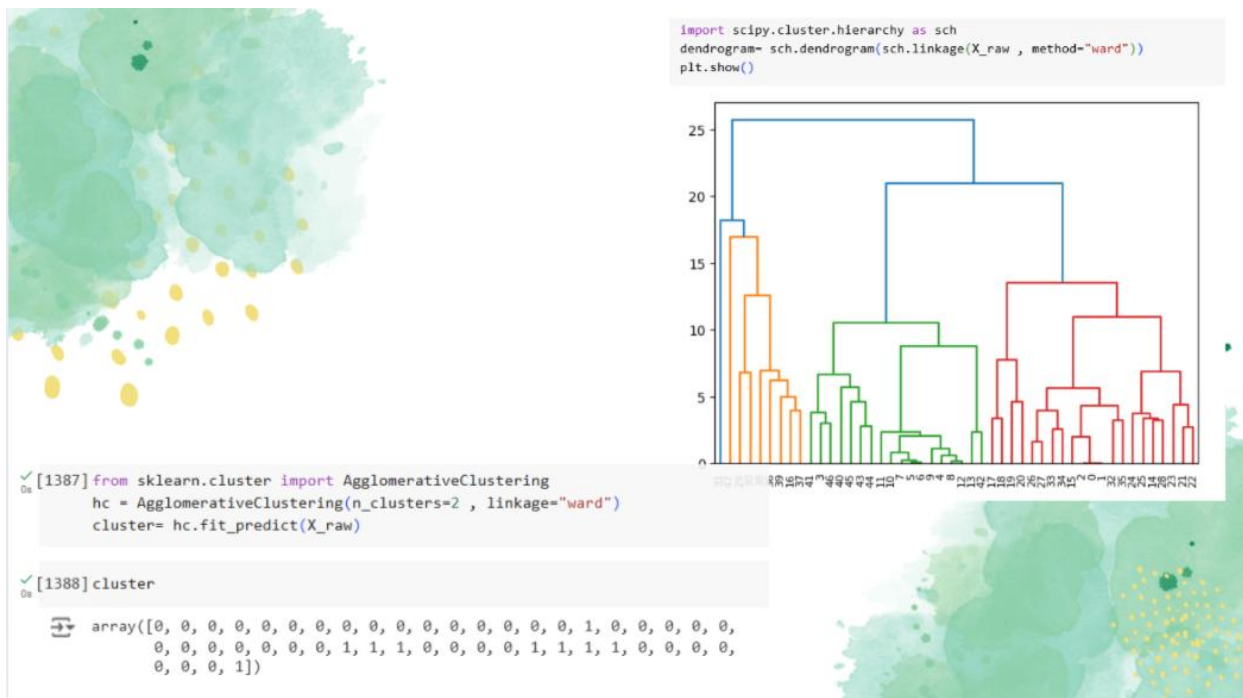
Number of NaNs: 0

[1384] from sklearn.preprocessing import StandardScaler
      sc = StandardScaler()
      X_raw = sc.fit_transform(X_raw)

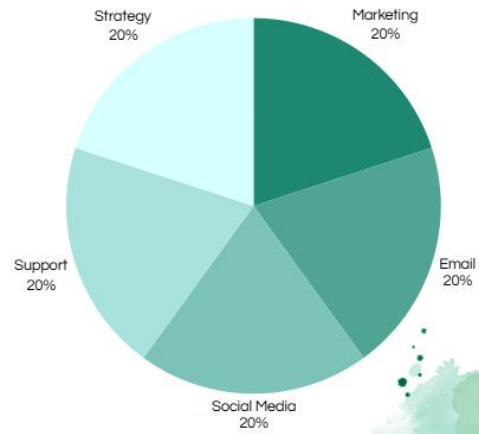
```

Extract the selected core features as a numeric matrix for preprocessing.

Fill missing values using nearby years (KNN). Prevents losing years due to NaNs.



Supervised Phase



Into Clustering phase I already
make copy for the same data
that i want to use it into supervised
phase



```
assert len(Supervised) == len(cluster), "Cluster length != rows in Supervised"
```

```
] Supervised = Supervised.sort_values("Year").reset_index(drop=True).copy()  
Supervised["Hot/Cold Year"] = cluster  
Supervised["regime_t1"] = Supervised["Hot/Cold Year"].shift(-1)  
Supervised = Supervised.iloc[:-1].copy()
```

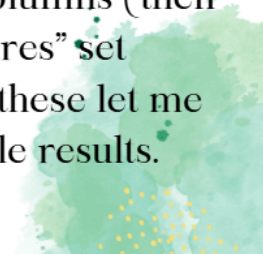
```
] feat = core_cols + ["Hot/Cold Year"]  
X = Supervised.loc[:, feat].to_numpy(dtype=float)  
y = Supervised["regime_t1"].astype(int).to_numpy()
```

Here i match the values of Cluster
with Supervised list by rows of year
and get column For prediction
"regime_t1"



Cases of Data

I tried many setups to get a strong, stable accuracy. The dataset has one big weak spot: 1967–1986 has lots of NaNs. So I tested simple cases: (1) drop those years and train on the clean years only, (2) keep all years but use only clean, reliable columns (then impute/scale), and (3) use a small “strong features” set (inward/outward/domestic totals, etc.). Comparing these let me choose the setup that gives the best and most stable results.



Data Used	Feature selection	Clusters (k)	Best model & score
Case 1 : the data is cut because most of rows nan 1967-1986	talk 71 column with have less nan "without care about name of feature"	2	Logistic Accuracy= 0.88 all another model bad "change test size dose not make sense"
Case 2: Full data 1967-2016	threshold way (19 cols) it also talk feature by method threshold "without care about name of feature"	3	Logistic $\approx$ 0.66
Case 2: Full data 1967-2016	Same way of upper stricter feature selection threshold (8 cols)	3	Decision Tree $\approx$ 0.8667, KNN $\approx$ 0.80
Case 3: "Strong-cols" (regex) Full data 1967-2016	Domain patterns (inward/outward/domestic values 8 counts + Asia/Africa); kept ~31 strongest "Focus on name of feature"	2	Decision Tree up to ~0.90, KNN ~0.80 Tree drops to ~0.67 when test size $\uparrow$; KNN more stable